Regression, Generalisation & Model evaluation

Session 3

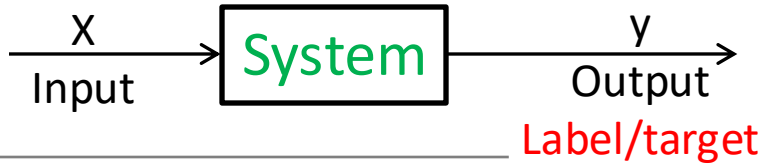
Applied Machine Learning

Yuhua Li
liy180@cardiff.ac.uk

Contents

- Linear regression
- Model training
- Polynomial regression
- Generalisation, overfitting, underfitting
- Regularisation
  - Regularized linear models
- Logistic regression
- Model Evaluation

Machine Learning Approaches: recap



- Supervised learning: learning with a labelled training dataset.

  For example, malware detection with training set of already labelled software applications.
  - Classification: where target is categorical
  - Regression: where target is continuous

- Unsupervised learning: discovering patterns in unlabelled dataset.

  For example: clustering user groups on social network based on their shared interests.
  - Clustering
  - Anomaly detection

- Reinforcement learning: learning based on feedback or reward

  For example: learning to play chess by wining or loss

Supervised Learning

- Regression (for continuous output)
  - Linear regression, polynomial regression
    - Ridge regression, LASSO, elastic net
  - Fuzzy logic systems (FL)
  - Artificial neural networks (ANN)
    - deep learning
  - …
- Classification (for categorical output)
  - Logistic regression
  - Support vector machines (SVM)
  - Bayesian network
  - K nearest neighbours (K-NN)
  - Decision tree (DT)
  - Ensemble learning
  - …



$y = f(X)$ (aka: label, target, Gold, Ground truth)

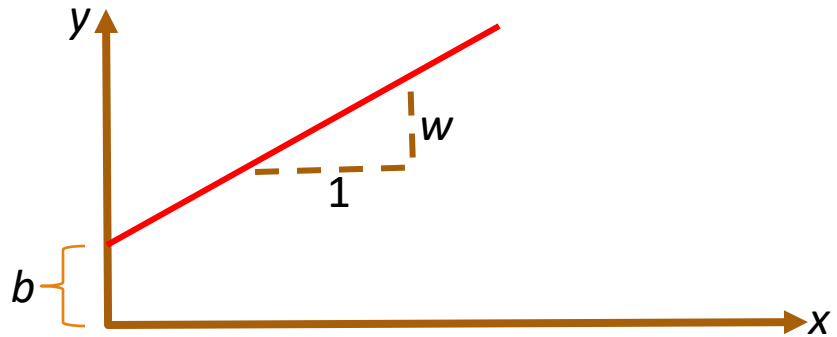
- Note many techniques work for both classification and regression problems, e.g., FL, ANN, k-NN, DT, SVM.

Linear Regression

- A linear model is a sum of weighted variables that predicts a target output value given an input data instance.

$$y = wx + b$$

w – slope/gradient, b – intercept



Linear Regression

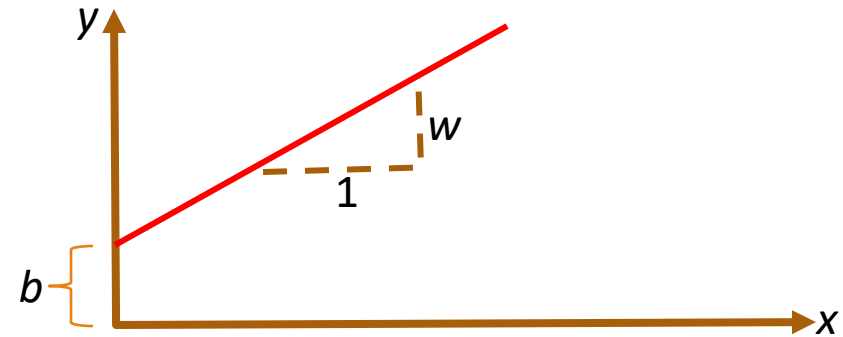
- A linear model is a sum of weighted variables that predicts a target output value y given an input data instance x.

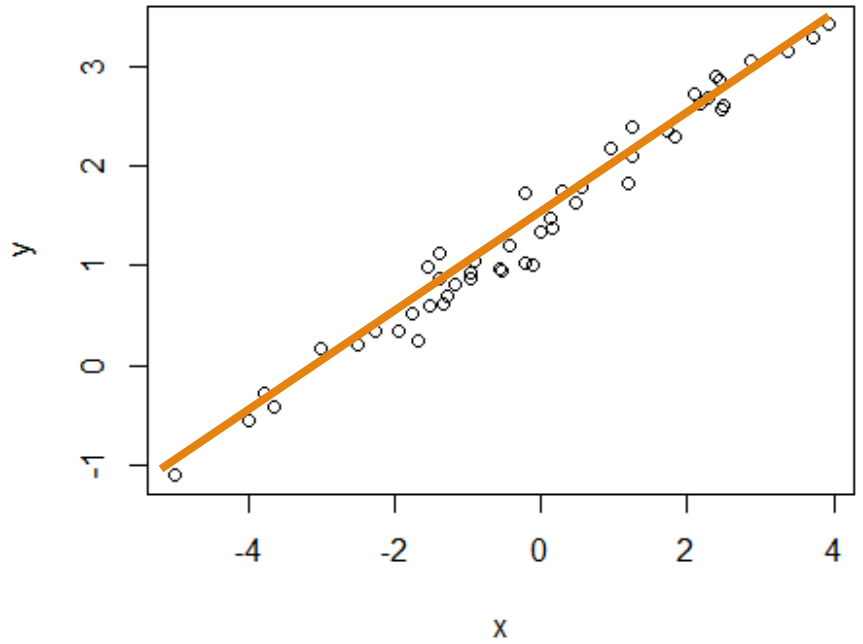
$$y = wx + b$$

w – slope/gradient, b – intercept

or for pairs of input-output (x_i, y_i) of *N* points

$$y_i = wx_i + b$$
$$i = 1, \cdots, N$$





Linear Regression

- In general form, for n features/variables of *N* points:

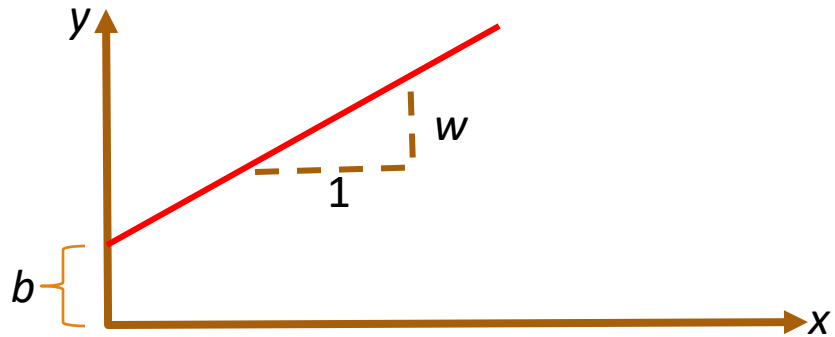
$$y_i = w_1 x_{i,1} + w_2 x_{i,2} + \cdots w_n x_{i,n} + b$$

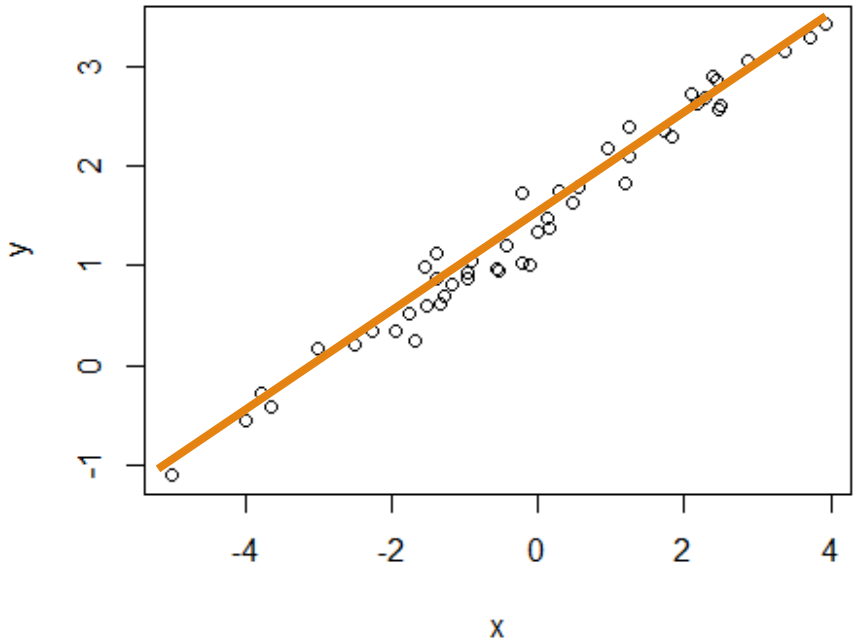
$$= \mathbf{w}^T \mathbf{x_i} + b$$

$$i = 1, \cdots, N$$

$w_1, w_2, \cdots w_n$ – feature weights, b – bias

- Linear regression is to find a linear model for a given data set.





Least-Squares Linear Regression
(Ordinary Least Squares)



$y = f(X) = w^T X + b$

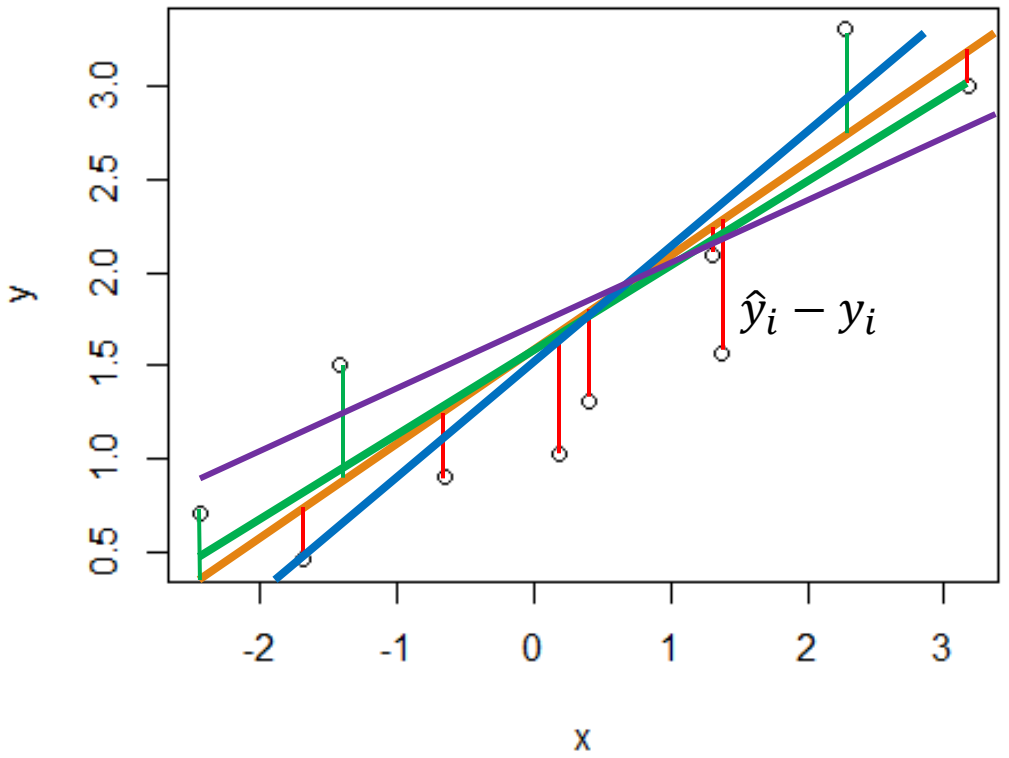
- Finds *w* and *b* that minimizes the mean squared error (MSE) of the linear model: the average of the sum of squared differences between predicted target $\hat{y}$ and actual target y values

$$L(w,b) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

$$= \frac{1}{N}\sum_{i=1}^{N}((\mathbf{w}^T\mathbf{x}_i + b) - y_i)^2$$

L refers to the objective function (aka, loss function, cost function, error function) of the model

- Finding the optimal model parameters $\boldsymbol{w} = (w_1, w_2, \cdots w_n)^T$ and b on a given dataset is called **training** (aka learning).



Contents

- Linear regression
- **Model training**
- Polynomial regression
- Generalisation, overfitting, underfitting
- Regularisation
  - Regularized linear models
- Logistic regression
- Model Evaluation

Model training

- Ways of training
  - Closed-form solution
  - Gradient descent
- Controlling model complexity

Training of Linear Regression

- Parameters of a linear regression model can be set by training the model to best fit the training dataset
- Two very different ways to train linear regression model
  - Using a direct “closed-form” equation through mathematical derivation → exact solution
  - Using an iterative optimization approach, called Gradient Descent (GD) → approximate solution

Linear Regression Training - closed form

- Include *b* as a parameter, $w_0 = b$, i.e., $\boldsymbol{w} = (w_0, w_1, w_2, \cdots w_n)^T$, we can rewrite the equation

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

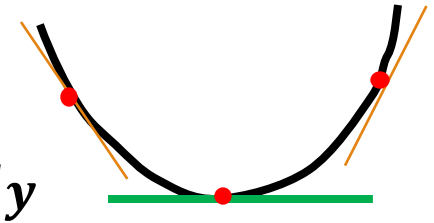
$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

$$= \frac{1}{N}\sum_{i=1}^{N}(\mathbf{w}^T \mathbf{x}_i - y_i)^2$$

- Parameters that minimize $L(w)$ can be obtained by setting its gradient to zero:

$$\nabla_w L(w) = 0$$

gives: $\hat{\boldsymbol{w}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$



The above is called normal equation, where each row of *X* is a point, *y* is the column vector of target values

- Computational complexity

  Involve matrix inverse with complexity of $O(n^{2.4})$ to $O(n^3)$,

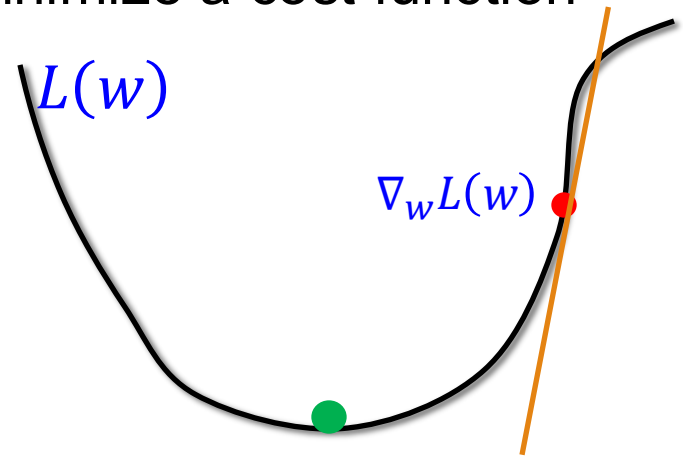
  where *n* is the number of features

- So computation is slow when *n* is large

year	month	day	hour	DEWP	TEMP	PRES	cbwd	Iws	Is	Ir	pm2.5
2010	1	4	2	-17	-10	1024	NW	29.95	0	0	25
2010	1	4	3	-18	-11	1024	NW	39.78	0	0	26
2010	1	4	4	-19	-11	1025	NW	48.72	0	0	28
2010	1	4	5	-20	-12	1026	NW	55.87	0	0	26
2010	1	4	6	[illegible]	-12	[illegible]	NW	64.81	0	0	20
2010	1	4	7		-13	[illegible]	NW	73.75	0	0	29
2010	1	4	8	[illegible]	[illegible]	1028	NW	80.9	0	0	26
2010	1	4	9	-22		1029	NW	90.73	0	0	27
2010	1	4	10	-22		1030	NW	100.6	0	0	[illegible]
2010	1	4	11	[illegible]	[illegible]	1031	NW	108.6	0	0	[illegible]
2010	1	4	12		-11	[illegible]	NW	117.6	0	0	[illegible]
2010	1	4	13	[illegible]	-10	[illegible]	NW	127.4	0	0	32
2010	1	4	14	-21	-10	1030	NW	136.3	0	0	28
2010	1	4	15	-21	-9	1030	NW	145.3	0	0	29
2010	1	4	16	-21	-9	1031	NW	152.4	0	0	30
2010	1	4	17	-20	-11	1032	NW	159.6	0	0	30

Linear Regression Training – Gradient Descent

- Gradient Descent is to tweak parameters iteratively in order to minimize a cost function e.g., $L(w) = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$ for linear regression.



- Procedure of gradient descent

  Initialise parameters *w* with random values

  Repeat

    Calculate current value of the loss function $L(w)$

    Calculate gradient of the loss function at current step $\nabla_w L(w)$

    Update *w* using current gradient

$$w^{next} = w^{current} - \eta \nabla_w L(w)$$

  until satisfying termination condition

where η is the learning rate which determine the step size of updating parameter w, and $\nabla_w L(w) = \frac{2}{N} X^T \cdot (X \cdot w - y)$ is the gradient

Linear Regression Training – Gradient Descent

- As the loss function (MSE) of linear regression is convex, it doesn't have local minimum. Using Gradient Descent for training linear regression is guaranteed to approach arbitrarily close the global minimum.
- Gradient Descent scales well with the number of features; training a Linear Regression model when there are hundreds of thousands of features is much faster using Gradient Descent than using the Normal Equation.
- A round of training over the whole training set is called an epoch.

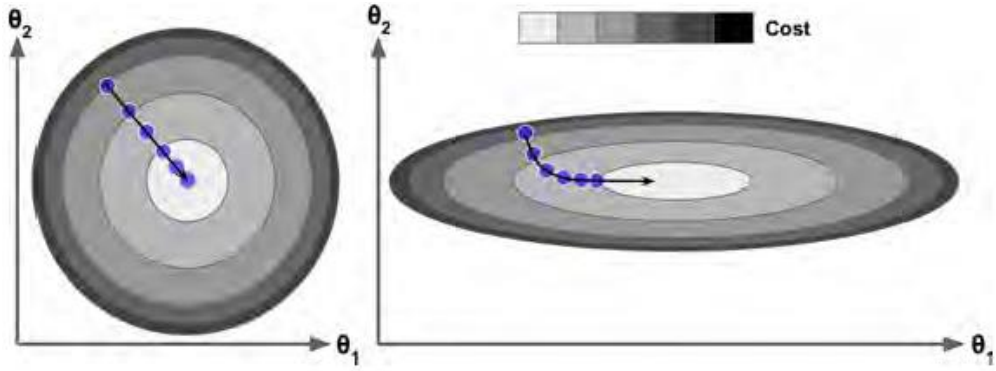
Gradient Descent: Tips

- Data normalization
  - When using Gradient Descent, you should ensure that all features have a similar scale
  - Normalization: scaled to fall within a small, specified range such as –1.0 to 1.0, 0.0 to 1.0 or $\mathcal{N}(0, 1)$
  - min-max normalization
    - For a variable *X*, $x \in X$, the range of X is is normalized from its original range $[min_X, max_X]$ to new range $[a, \ b]$

$$x' = \frac{b - a}{max_X - min_X}(x - min_X) + a$$

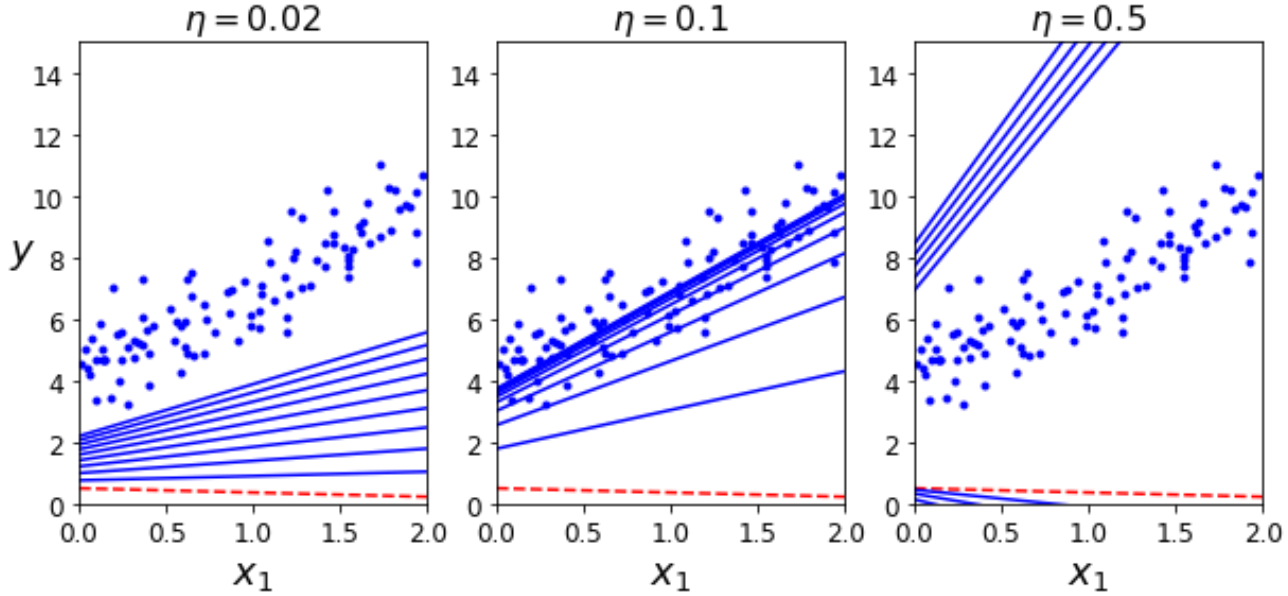
  - z-score normalization

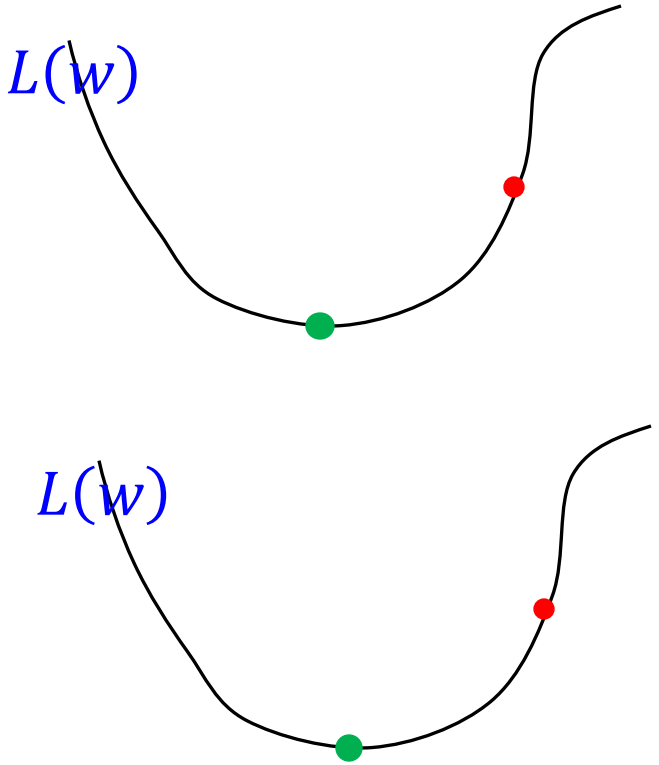
$$x' = \frac{x - mean_X}{std_X}$$



Gradient Descent: Tips

- Leaning rate
  - Too small: take long time to reach the solution
  - Too big: may not converge to the solution, overshot



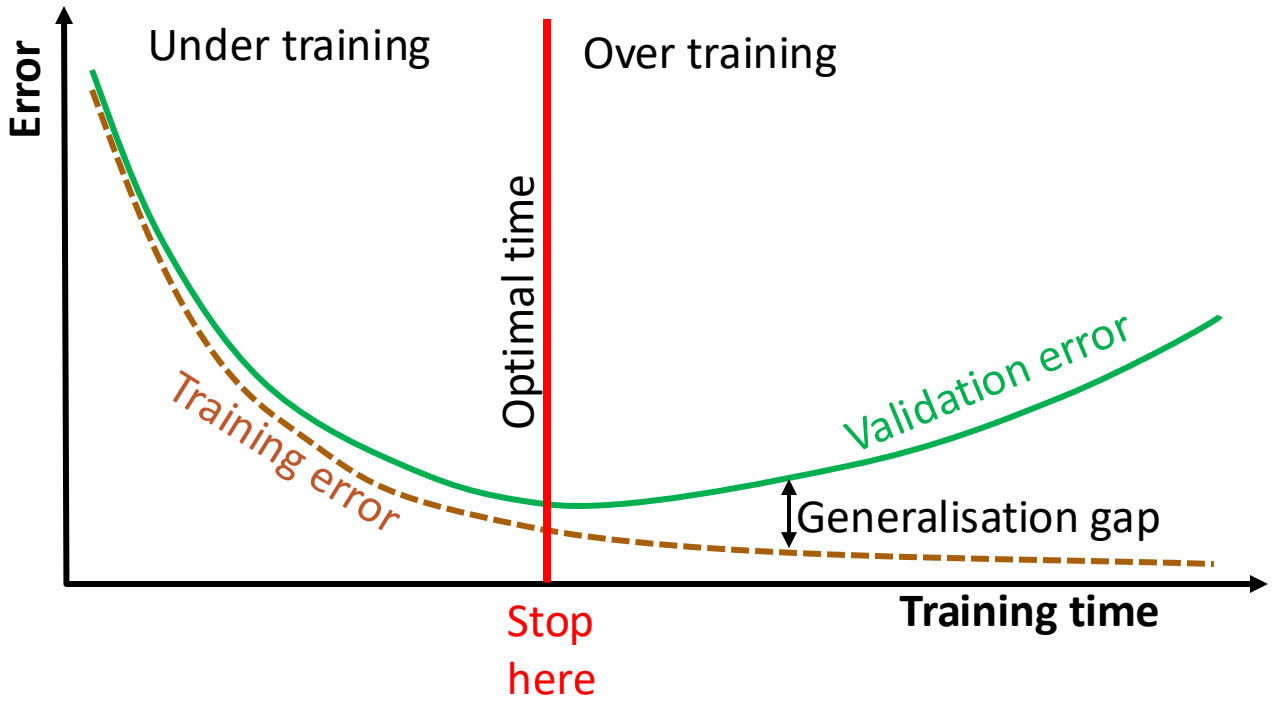


  - Learning schedule: use a function to update learning rate, i.e., start with a large learning rate and gradually decrease it at each step t

$$\text{e.g., } \eta(t) = \frac{a}{b + t}$$

Gradient Descent: Tips

- Early stopping
  - Training can be terminated if the value of the loss function falls below pre-set values or the number of repetitions reaches the maximum number of iterations.
  - Use a validation set, stop training when the performance on validation set start decreasing or no improvement after a set number of iterations.

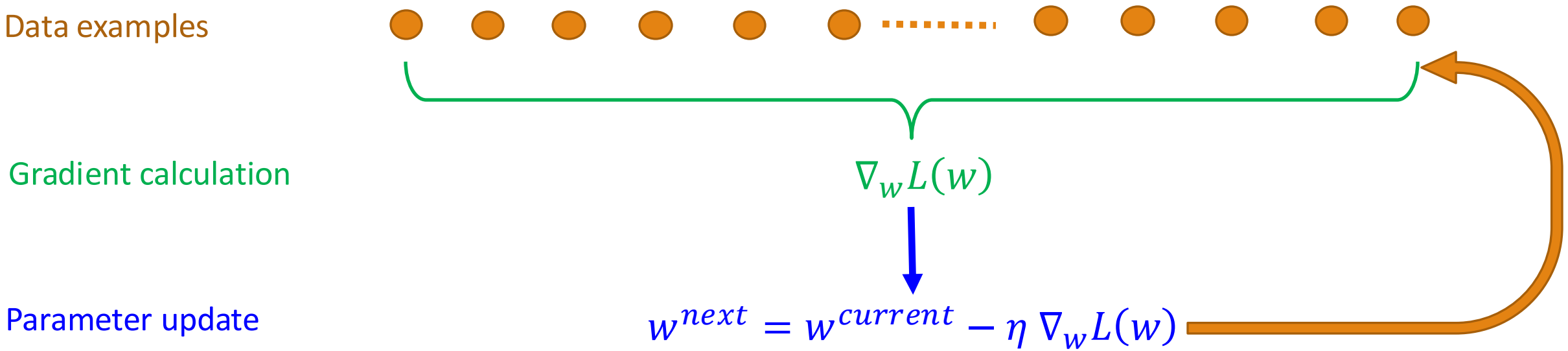


Ways of gradient decent

- There are 3 ways to calculate gradient of the cost function and update parameters depending how often this is done
  - Batch Gradient Descent
  - Stochastic Gradient Descent
  - Mini-batch Gradient Descent

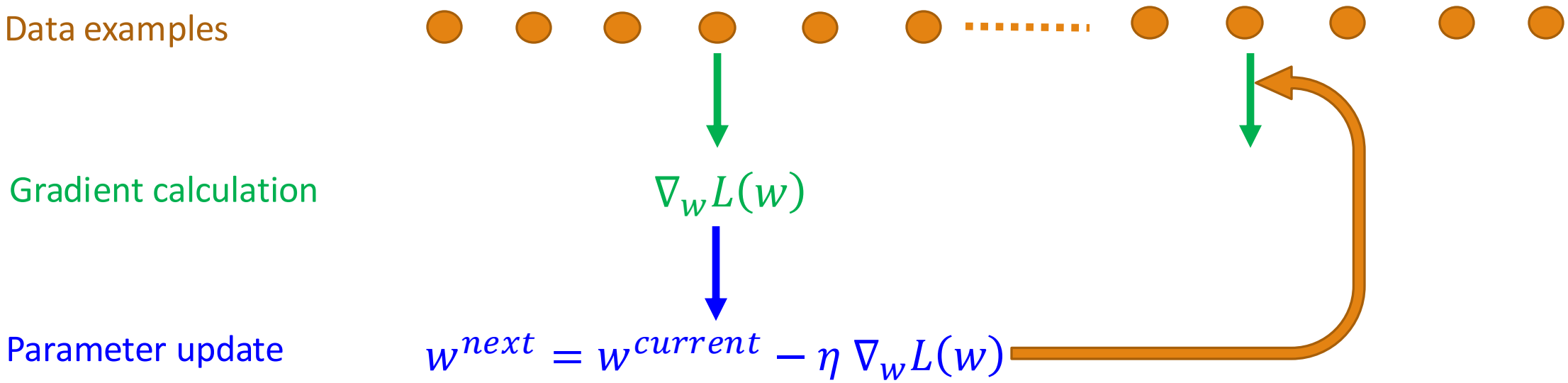
Batch Gradient Descent

- uses the whole batch of training dataset X to calculate the cost function and its gradient at every step
- Slow updating when training set is large



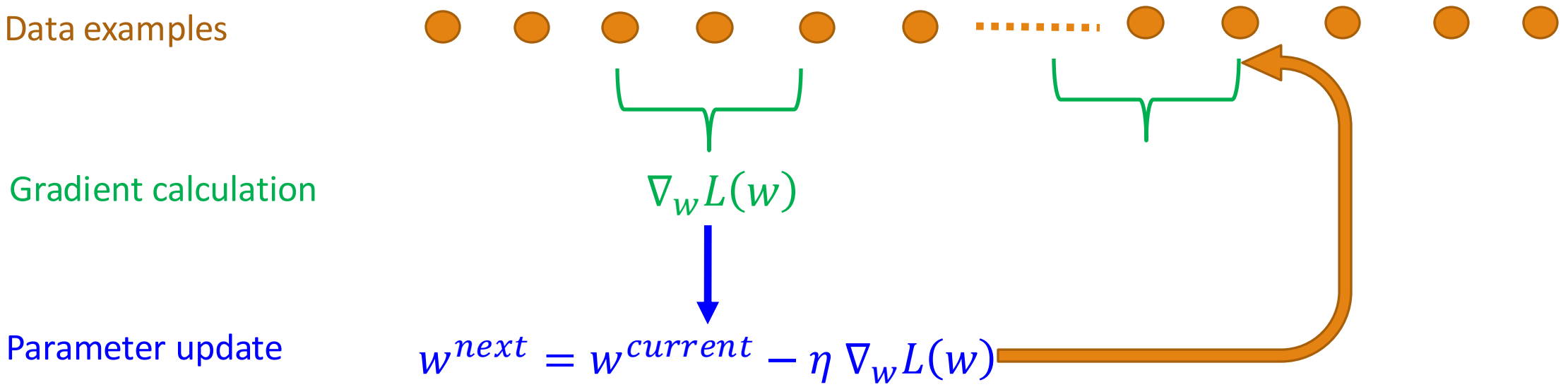
Stochastic Gradient Descent

- Picks a random instance in the training set at every step and computes the gradients based only on that single instance.
- Frequent updating



Mini-batch Gradient Descent

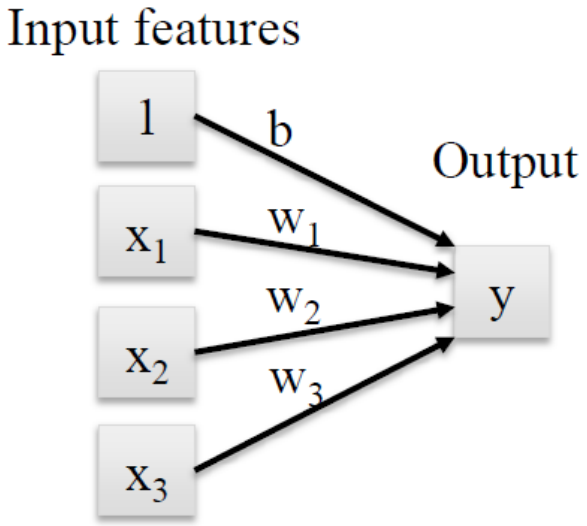
- At each step, instead of computing the gradients based on the full training set (as in Batch GD) or based on just one instance (as in Stochastic GD), Mini-batch GD computes the gradients on small random sets of instances called mini-batches.



Computational graph of linear regression

- $y = w_1 x_1 + w_2 x_2 + \cdots w_n x_n + b$
- Rewrite as

$$y = b\,1 + w_1 x_1 + w_2 x_2 + \cdots w_n x_n$$

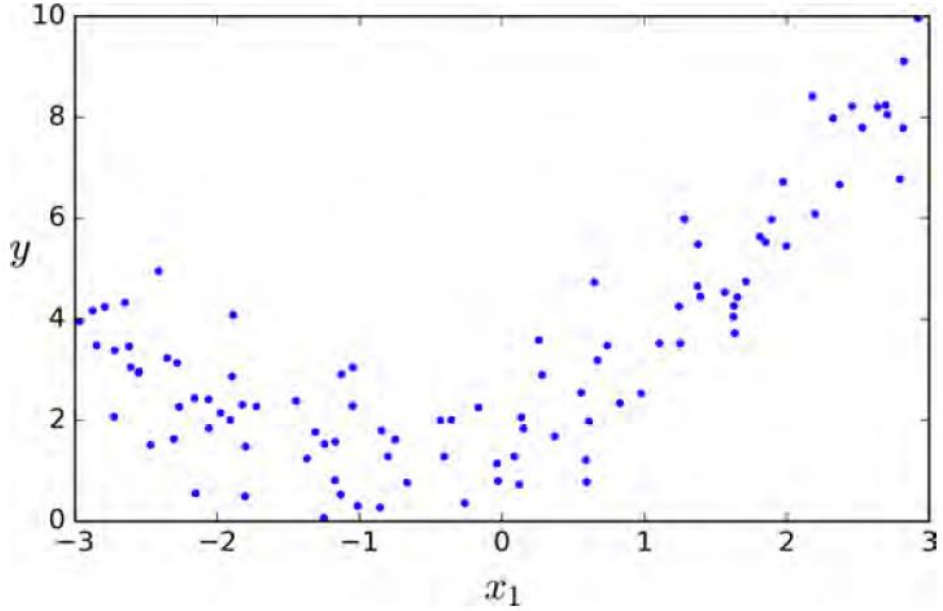


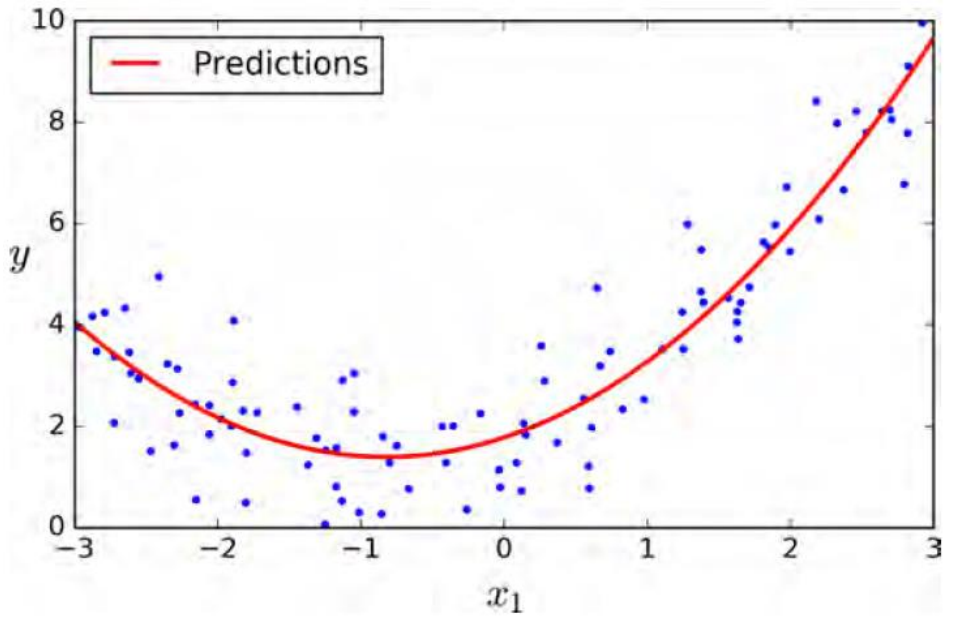
Contents

- Linear regression
- Model training
- **Polynomial regression**
- Generalisation, overfitting, underfitting
- Regularisation
  - Regularized linear models
- Logistic regression
- Model Evaluation

Polynomial regression

- Add powers of each feature (and multiplications of features) as new features, then train a linear model on this extended set of features





e.g., for 2nd degree of univariate: $y = w_0 + w_1 x_1 + w_2 x_1^2 + N(\mu, \sigma^2)$

In particular, for the figures above: $y = 2 + 1x_1 + 0.5x_1^2 + N(0, 1)$

So x_1^2 can be consider a new feature. In fact, each introduced term of polynomial can be treated as a new feature in ordinary linear regression. Thus training methods of ordinary linear regression can be used for training of polynomial regression.

Figs courtesy of Géron 2017.

Polynomial regression

- Which degree?

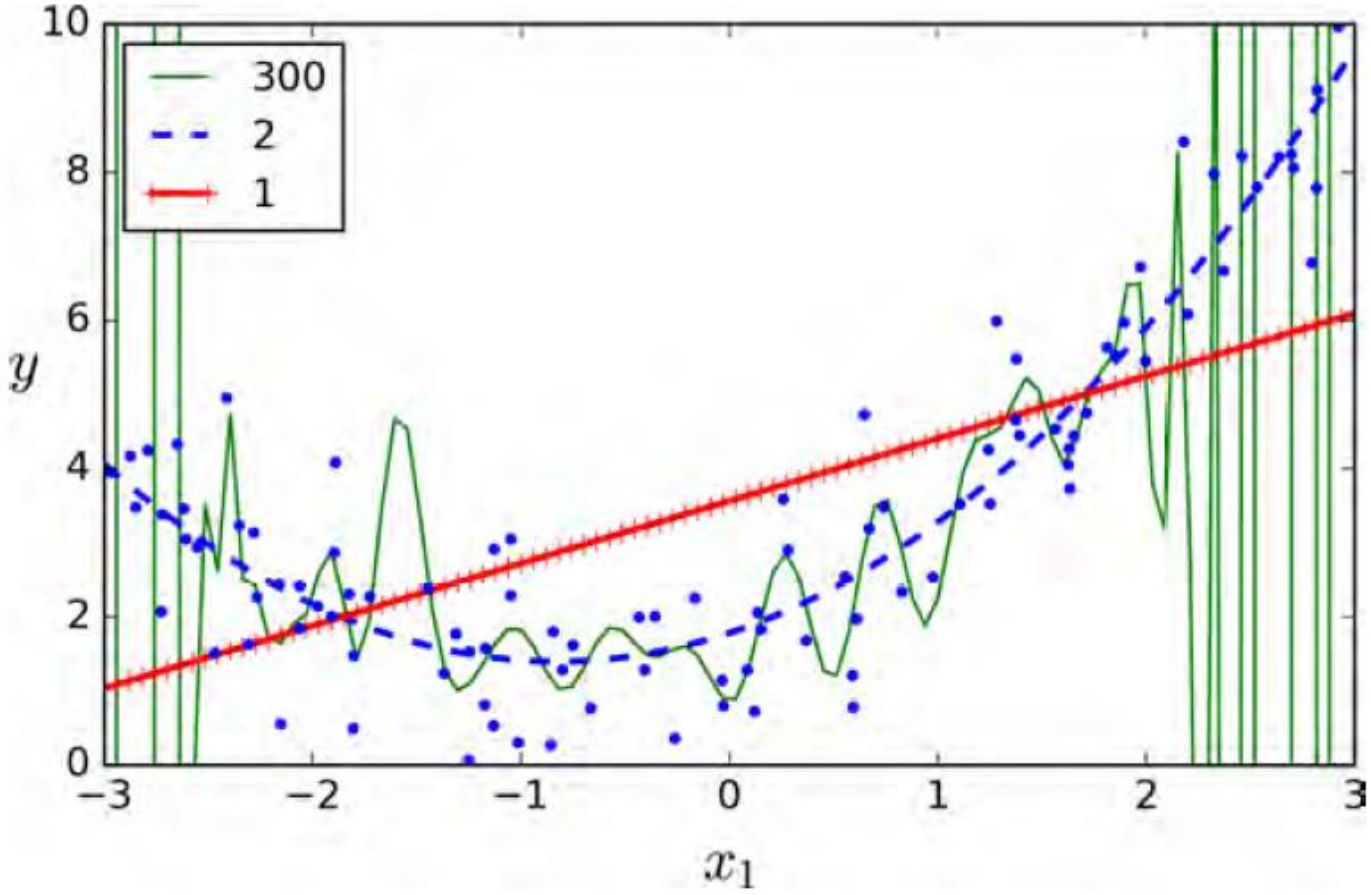


Fig courtesy of Géron 2017.

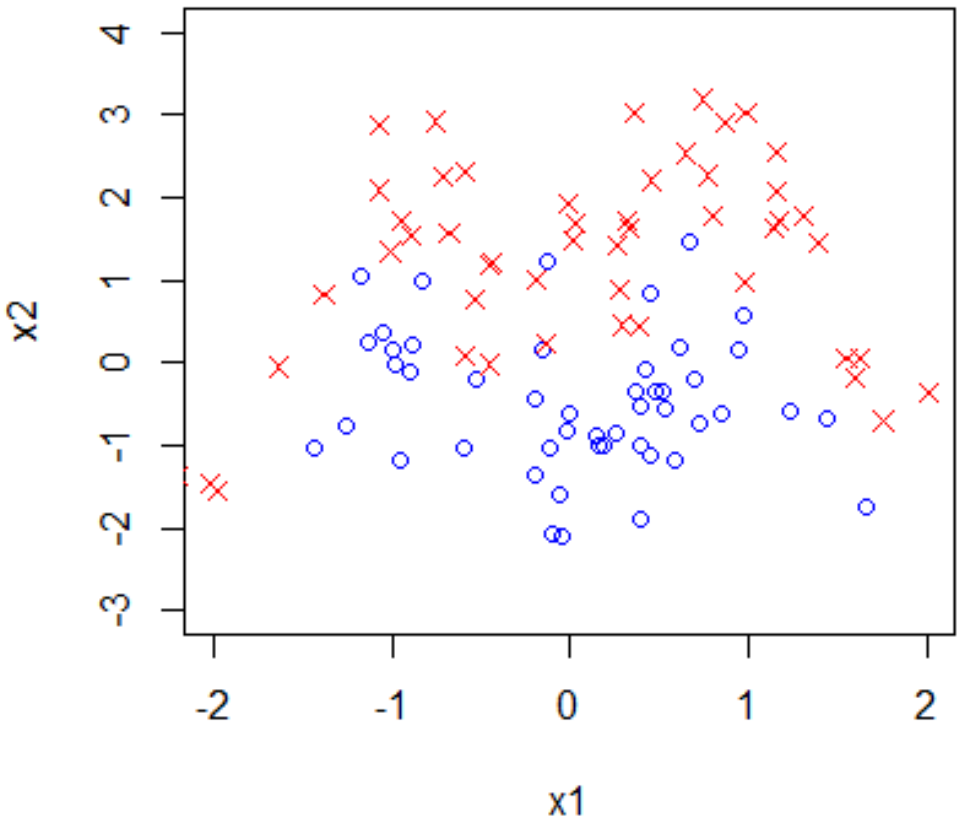
Contents

- Linear regression
- Model training
- Polynomial regression
- **Generalisation, overfitting, underfitting**
- Regularisation
  - Regularized linear models
- Logistic regression
- Model Evaluation
  - Metrics for Performance Evaluation
  - Performance assessment methods

Generalisation, overfitting, underfitting

- Generalisation is the ability to perform well on previously unseen data.

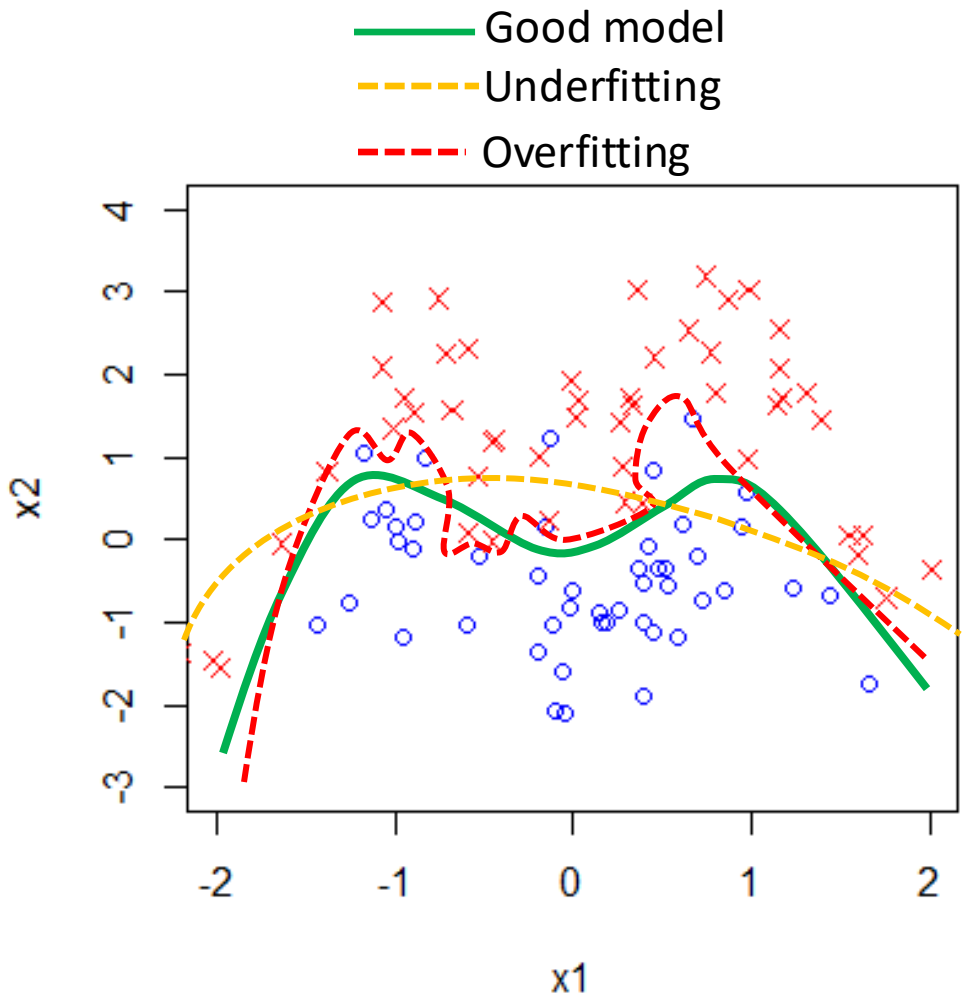
- The factors determining how well a learning algorithm will perform are its ability to
  - Make the training error small.
  - Make the gap between training and test error small.



Generalisation, overfitting, underfitting

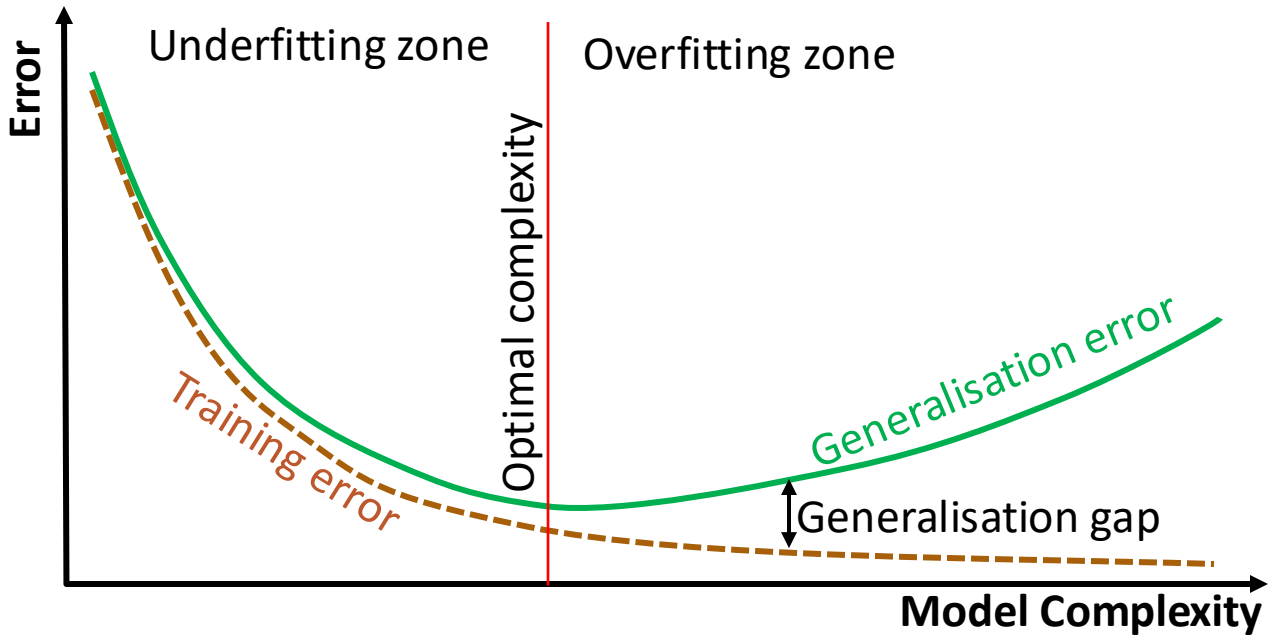
- Underfitting occurs when the model is not able to obtain a sufficiently low error on the training set.

- Overfitting occurs when the gap between the training error and test error is too large.



Generalisation, overfitting, underfitting

- Typical relationship between model complexity and error



Overfitting

- Overfitting symptom
  - High accuracy on training data, low accuracy on unseen data

- Ways to deal with overfitting
  - regularisation,
  - more training data,
  - dimensionality reduction,
  - dataset shift → model adaptation.

Contents

- Linear regression
- Model training
- Polynomial regression
- Generalisation, overfitting, underfitting
- **Regularisation**
  - Regularized linear models
- Logistic regression
- Model Evaluation

Regularisation

- Regularisation is any modification to a learning algorithm that is intended to reduce its generalisation error but not its training error.

- Regularization prevents overfitting by restricting the model, typically to reduce its complexity.

- A common way of regularisation is to add a penalty term to its original loss function, with respect to model parameters.

Regularized Linear Models

- Ridge regression
- LASSO
- Elastic net

l_p norm (p-norm) of a vector w: $\|w\|_p = (\sum_{i=1}^{n} |w_i|^p)^{1/p}$

Ridge Regression

- Adds L2 (Euclidean norm) penalty to the cost function of plain linear regression, i.e., Ordinary Least Squares
  - Ridge regression learns w, b using the same least-squares criterion but adds a penalty for large variations in w parameters

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{w}^T\mathbf{x}_i - y_i)^2 \boxed{+\alpha\sum_{j=1}^{p} w_j^2}$$

- The parameter $\alpha > 0$ controls amount of regularization (default = 1.0). Larger values specify stronger regularization.
- This kind of parameters like α in above equation is called hyperparameters, which need to be set by users before training process.
- The same learning algorithms (closed form or gradient descent) can be used to find model parameters $\boldsymbol{w}$.

LASSO regression

- Adds L1 (absolute value norm) penalty

  LASSO (least absolute shrinkage and selection operator)
  - Minimize MSE and the sum of the absolute values of the parameters

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{w}^T\mathbf{x}_i - y_i)^2 \boxed{+\alpha\sum_{j=1}^{p}|w_j|}$$

  - This has the effect of setting parameter weights in w to zero for the least influential variables. This is called a sparse solution: a kind of feature selection.

Elastic Net

- Adds L1 and L2 penalty

  Elastic net plays in the middle between ridge regression and LASSO

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{w}^T\mathbf{x}_i - y_i)^2 \; +\gamma\,\alpha\sum_{j=1}^{p}|w_j| + (1-\gamma)\,\alpha\sum_{j=1}^{p}w_j^2$$

  where $\gamma \in [0, 1]$ is L1 ration.

  When $\gamma = 0$, Elastic Net is equivalent to Ridge Regression

  When $\gamma = 1$, Elastic Net is equivalent to Lasso Regression

- Once the parameters are learned, the prediction formula for all ridge regression, LASSO regression and elastic net is the same as plain linear regression, i.e.,

  $\hat{y}_i = \mathbf{w}^T \boldsymbol{x}_i$ for a single point $\boldsymbol{x}_i$, or

  $\hat{\boldsymbol{y}} = \boldsymbol{X}\,\boldsymbol{w}$ for a set *X* (each row of *X* is a data point).

Which regression model?

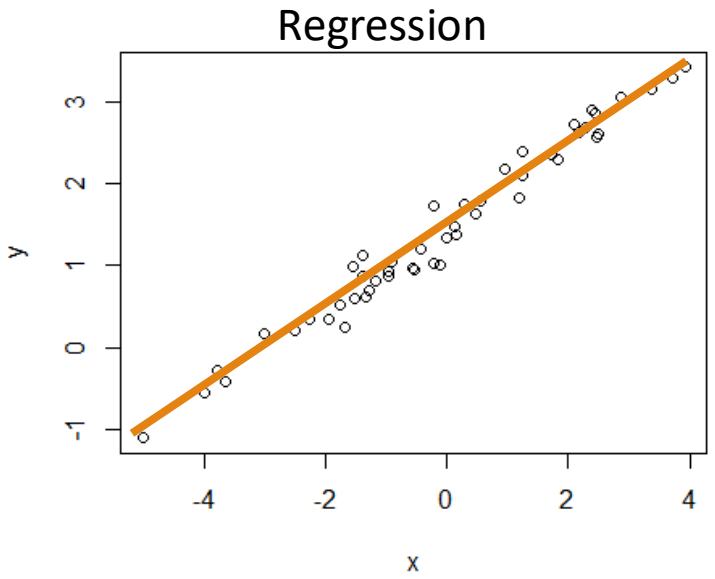
- Model should have appropriate capacity to approximate the data, regularisation is an effective way to control model complexity.
- Generally model incorporating regularization achieve better performance, so plain linear regression (no regularisation) should be avoided.
- Use ridge regression if all features are useful.
- Use LASSO if you suspect some/many features are not useful, so those features could be eliminated.
- Use elastic net if no intuition about feature usefulness is obvious. However, there are 2 hyperparameters, α and γ, need to be decided.
- Introduce polynomial terms for data with non-linear characteristics.

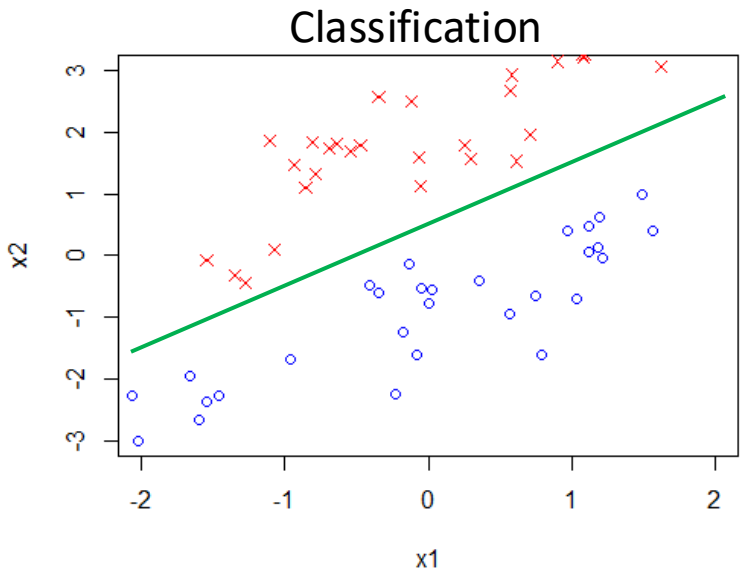
Contents

- Linear regression
- Model training
- Polynomial regression
- Generalisation, overfitting, underfitting
- Regularisation
  - Regularized linear models
- **Logistic regression**
- Model Evaluation

Logistic Regression

- Regression vs. classification



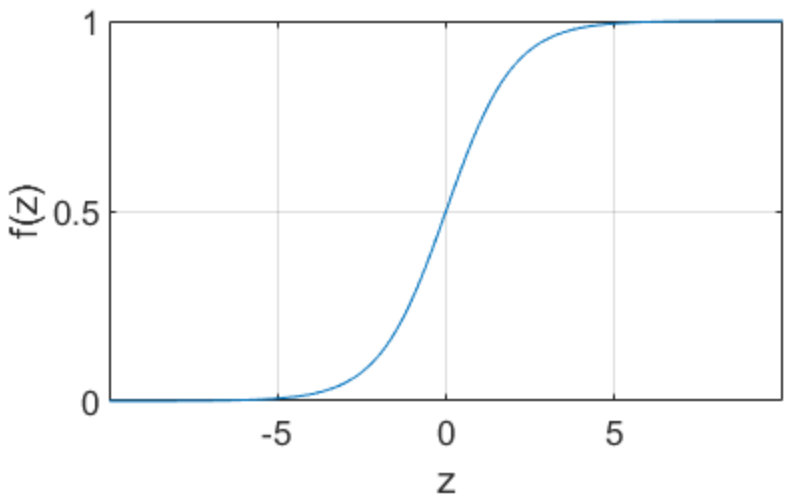


- Logistic regression is a linear model for classification
- It transforms the output of ordinary linear regression using logistic function, so given the name logistic regression

Logistic Regression

- Logistic function (aka sigmoid function)

$$f(z) = \frac{1}{1 + e^{-z}}$$



- Logistic Regression

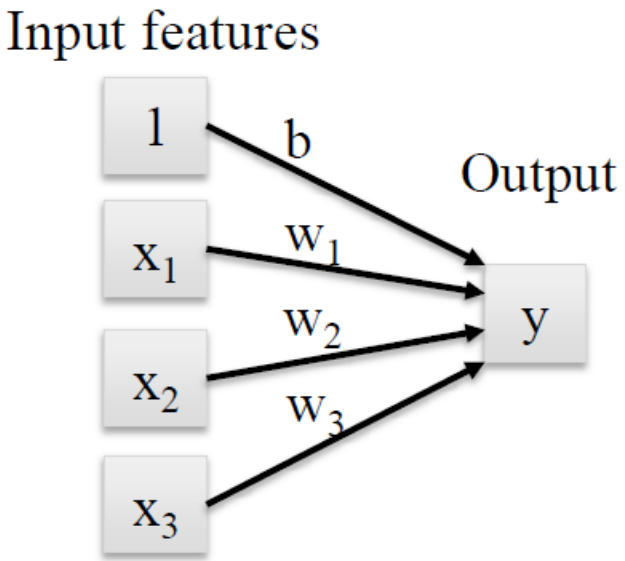
$$y = f(b + w_1 x_1 + w_2 x_2 + \cdots w_n x_n)$$

$$= \frac{1}{1 + e^{-(b + w_1 x_1 + w_2 x_2 + \cdots w_n x_n)}}$$

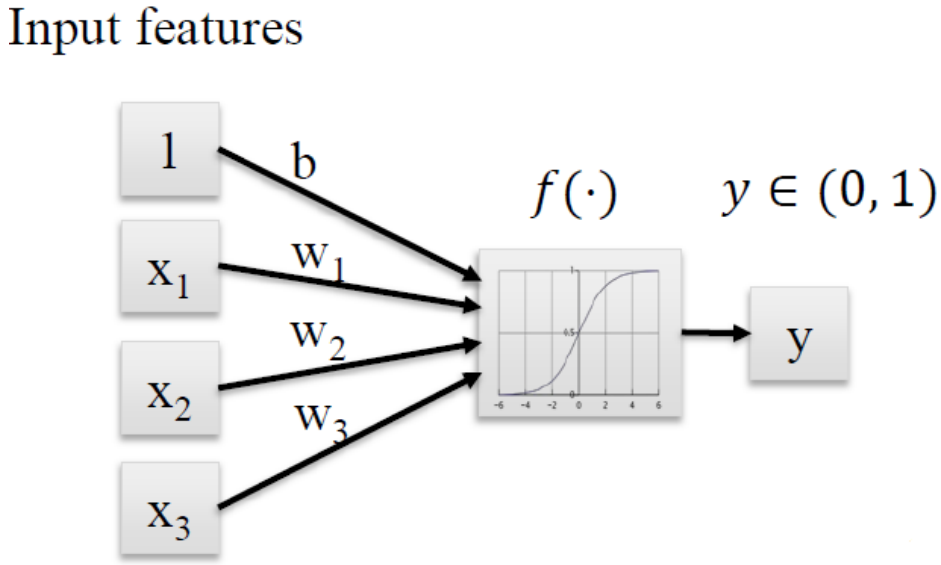
Logistic Regression

- The logistic function transforms real-valued input to an output number y between 0 and 1, interpreted as the probability the input object belongs to the positive class, given its input features $(x_1, x_2, \cdots x_n)$.
- If its output value is greater than a threshold (usually 0.5), then the object belongs to positive class, otherwise negative class.

Linear regression



Logistic regression



Logistic Regression Training

- Cost function (log loss, aka cross-entropy loss)

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log\left(\hat{p}^{(i)}\right) + \left(1 - y^{(i)}\right)log\left(1 - \hat{p}^{(i)}\right)\right]$$

- No known closed-form equation to compute the value of parameters that minimizes this cost function
- So gradient descent (BGD, SGD, mini-batch GD) is used to find parameters.
- The cost function is convex, so Gradient Descent (or any other optimization algorithm) is guaranteed to find the global minimum

Contents

- Linear regression
- Model training
- Polynomial regression
- Generalisation, overfitting, underfitting
- Regularisation
  - Regularized linear models
- Logistic regression
- **Model Evaluation**

Model Evaluation

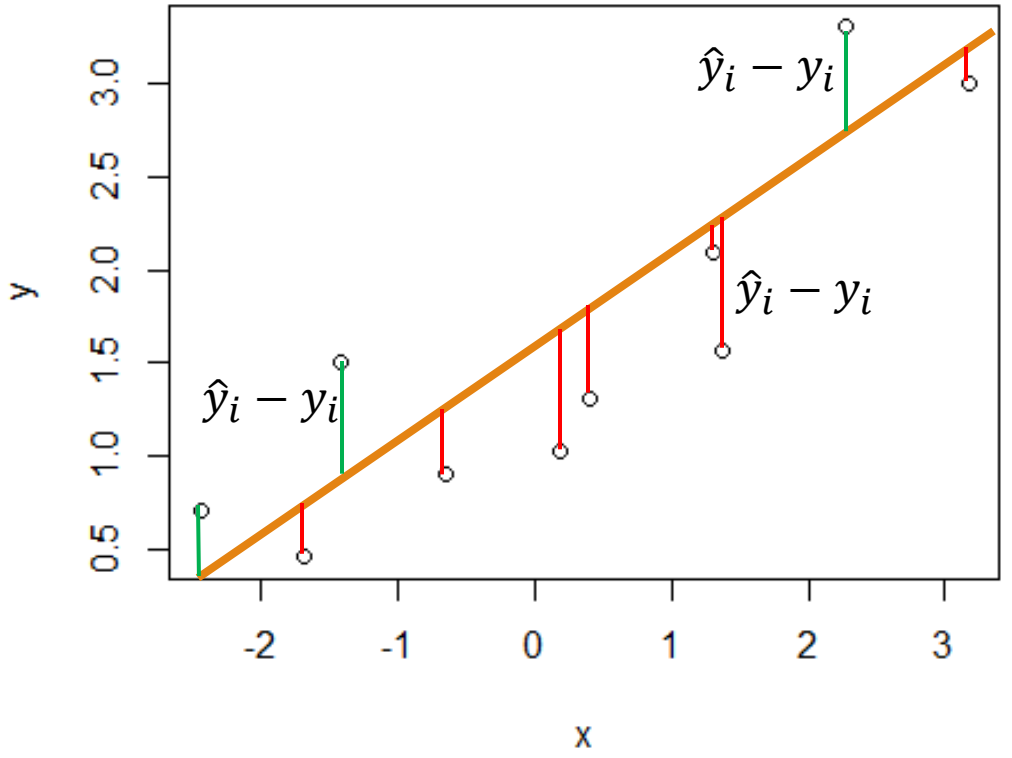
- Metrics for Performance Evaluation
  - Regression Performance Evaluation Metrics
  - Classification Performance Evaluation Metrics

- Methods for Performance Evaluation
  - How to evaluate the performance of a model?
  - How to obtain reliable estimates?

Regression Performance Evaluation Metrics

- Mean squared error: $MSE = \frac{1}{N}\sum_{i=1}^{N} e_i^2 = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$
- Root mean squared error: $RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2}$
- Mean absolute error: $MAE = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$
- Mean absolute percentage error:

$$MAPE = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{\hat{y}_i - y_i}{y_i}\right| \cdot 100\%$$



where y_i is the target value, $\hat{y}$ is the predicted target value, $e_i = \hat{y}_i - y_i$ is the error between the target and the predicted target for data point i, *N* is the total number of points.

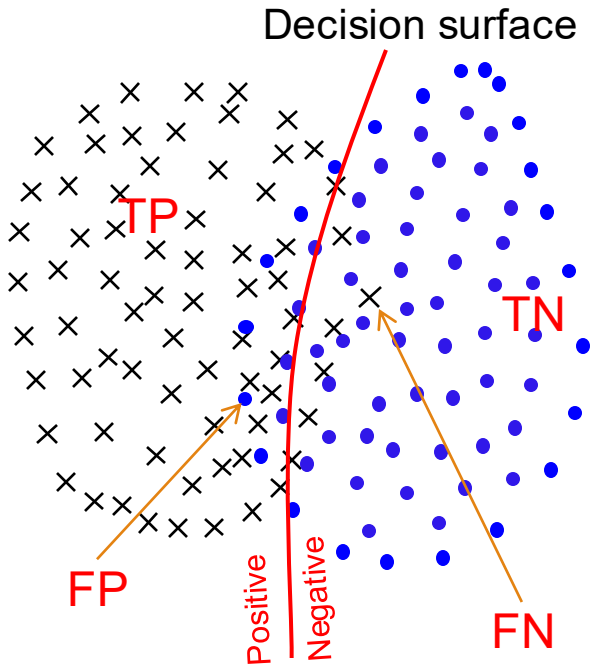
Classification Performance Evaluation Metrics

- Accuracy/error
- Recall, precision, F-measure
- ROC, AUC
- Hypothesis test
  - t-test, Wilcoxon signed-rank test, Kappa test

Metrics for Performance Evaluation

- Focus on the predictive capability of a model
  - Rather than how fast it takes to classify or build models, scalability, etc.
- Confusion Matrix:

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes (positive)	TP	FN
	Class=No (negative)	FP	TN



- True positives (TP): These refer to the positive points that were correctly labeled by the classifier.
- True negatives (TN): These are the negative points that were correctly labeled by the classifier.
- False positives (FP): These are the negative points that were incorrectly labeled as positive.
- False negatives (FN): These are the positive points that were mislabeled as negative.

Metrics for Performance Evaluation…

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	TP	FN
	Class=No	FP	TN

- Most widely-used metric:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Other performance Measures

False positive rate: $\text{fp rate} = \frac{FP}{N}$

True positive rate: $\text{tp rate} = \frac{TP}{P}$

	PREDICTED CLASS			Total points
ACTUAL CLASS		Class=Yes	Class=No	
	Class=Yes	TP	FN	P
	Class=No	FP	TN	N

- **Precision** can be thought of as a measure of exactness (i.e., what percentage of points labeled as positive are actually such)

$$\text{Precision (p)} = \frac{TP}{TP + FP}$$

- **Recall** is a measure of completeness (what percentage of positive points are labeled as such)

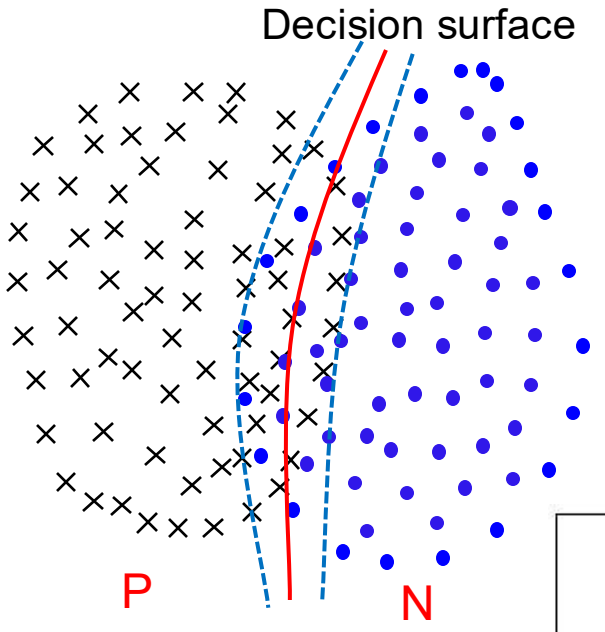
$$\text{Recall (r)} = \frac{TP}{TP + FN}$$

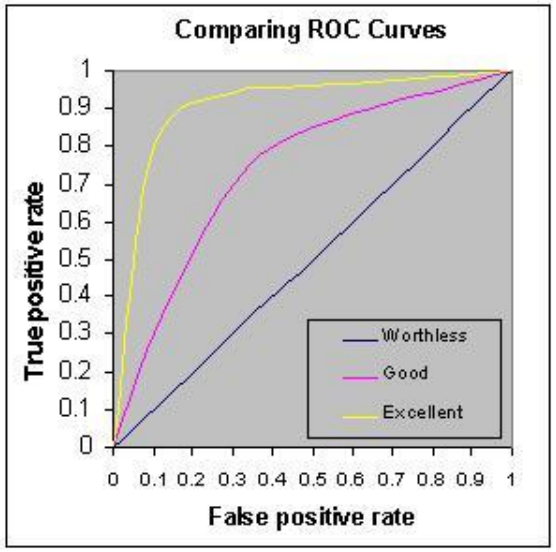
- **F–measure** is a combination of precision and recall

$$\text{F - measure (F)} = \frac{2 \times precision \times recall}{precision + recall} = \frac{2p \times r}{p + r}$$

Receiver Operating Characteristics (ROC) curve

- ROC courves are two-dimensional graphs in which *tp* rate is plotted on the Y axis and *fp* rate is plotted on the X axis. An ROC curve depicts relative tradeoffs between benefits (true positives) and costs (false positives).
- It is a technique for visualizing, organizing and selecting classifiers based on their performance.
- It is especially useful for domains with skewed class distribution and unequal classification error costs.
- The area under an ROC curve (**AUC**) is a performance measure that reduces an ROC curve to a single scalar value representing expected performance.





Limitation of Accuracy

- Consider data imbalance in a 2-class problem
  - Number of Class 0 examples = 9990
  - Number of Class 1 examples = 10
    - If model predicts everything to be class 0, accuracy is 9990/10000 = 99.9 %
      - Accuracy is misleading because model does not detect any class 1 example
- For imbalanced data, use confusion matrix, precision, recall, or F measure to determine class wise performance of the classifier.
- Data imbalance can be better dealt with by
  - using undersampling or oversampling (e.g., SMOTE) to make the data balanced
  - altering the prediction threshold value by doing probability caliberation and finding a optimal threshold using AUC-ROC curve
  - Using cost-sensitive learning, assigning weight to classes such that the minority classes gets larger weight
  - Using novelty/anomaly detection

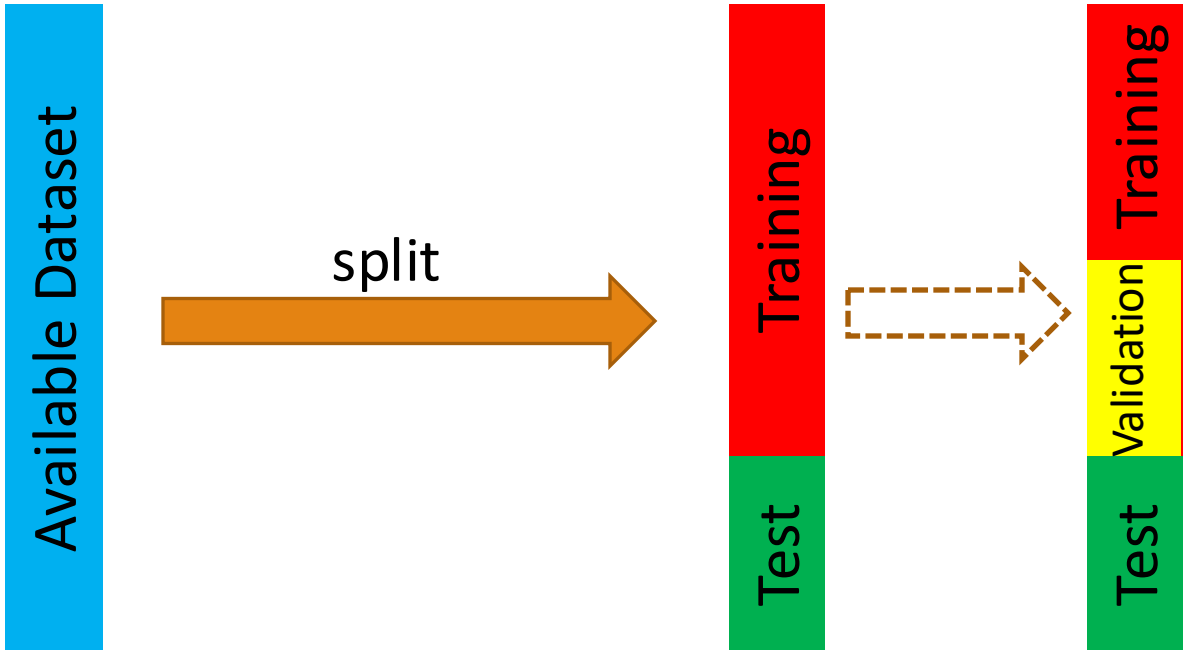
Methods for Performance Evaluation

- How to obtain a reliable estimate of performance?

- Performance of a model may depend on other factors besides the learning algorithm:
  - Class distribution
  - Cost of misclassification
  - Size of training and test sets

Performance assessment methods

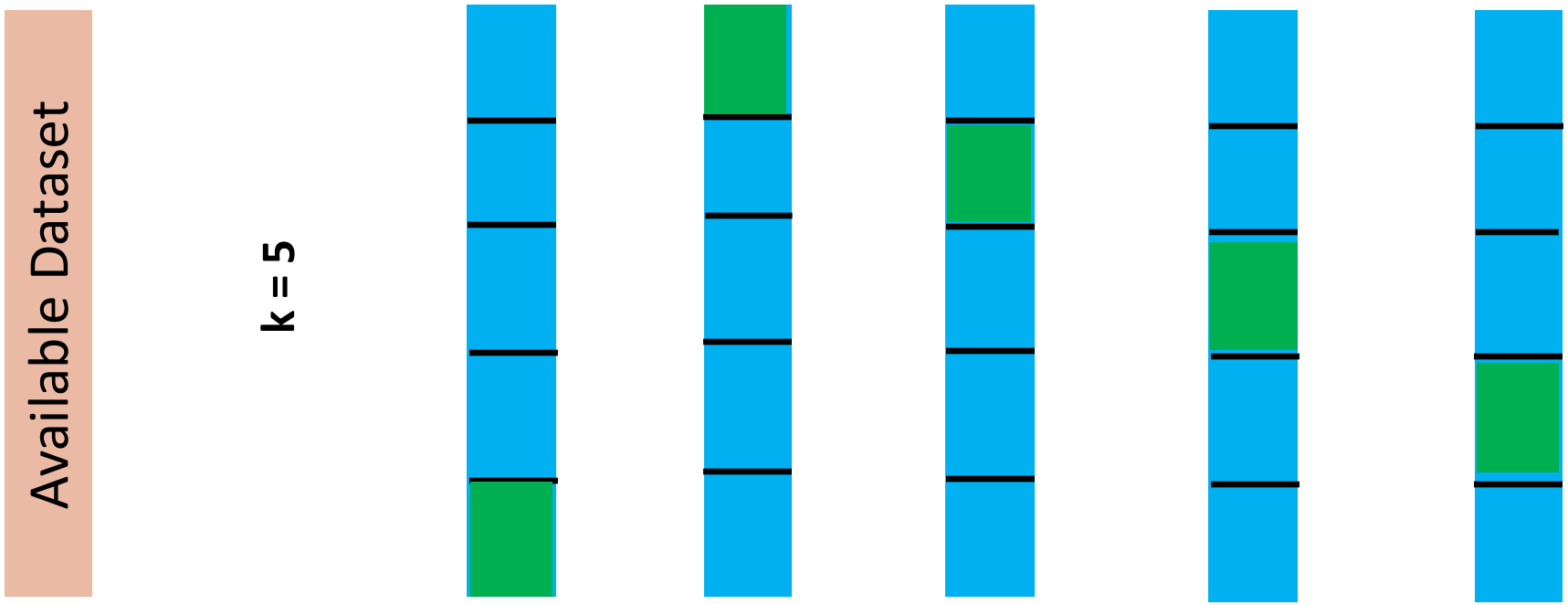
- Holdout
  - Roughly reserve 2/3 for training and 1/3 for testing. However, this ratio can be differently depending on the size of available data and training time.



- Random subsampling
  - Repeated holdout

Performance assessment methods

- Cross validation
  - Partition data into k disjoint equally-sized subsets
  - k-fold: train on k-1 partitions, test on the remaining one. Commonly k = 10.
  - Leave-one-out: k=N, i.e., only one point in each fold



Performance assessment methods: sampling

- Stratified sampling
  - If the given dataset ***D*** is divided into mutually disjoint parts called strata, a stratified sample of ***D*** is generated by obtaining a simple random sample (SRS) at each stratum. This helps ensure a representative sample, especially when the data are skewed.

- Bootstrap
  - Bootstrap method samples the given dataset ***D*** uniformly with replacement, i.e., each time a data point is selected, it is re-added to the given dataset ***D*** so it is equally likely to be selected again.

Reading

- Aurélien Géron. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition, 2022.
  Chapter 4

- Christopher Bishop, Hugh Bishop. Deep Learning: Foundations and Concepts. Springer, 2024.
  Chapter 7

Or

- Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.
  Sections 4.3, 5.1-5.3